Charotar University of Science and Technology

Chandubhai S. Patel Institute of Technology (CSPIT), CHARUSAT

Department of Information Technology / Computer Engineering

ITUE301: Advanced Web Development Frameworks

Open-Book Practical Examination | B.Tech. Semester 5 | AY 2026–27

Date	24 / 08 / 2026
Branch	CE / CSE / IT / AIML
Duration	2 Hours
Total Marks	20 Marks (4 Marks × 5 Tasks)
Tech Stack	React (frontend) + Express.js (backend) + MongoDB with Mongoose
Internet	Permitted for official documentation, MDN, Stack Overflow reference.
AI Tools	Permitted: ChatGPT, GitHub Copilot, Claude, Gemini, etc. may be used. Students must be able to explain every line of submitted code during viva.
Code Sharing	Strictly prohibited: identical submissions across students will be awarded zero.

General Instructions

1. Read the complete question paper before starting implementation. Plan your folder structure first.
2. Attempt all five tasks in your allotted set. Partial implementation earns partial marks: do not leave any task blank.
3. Use React for frontend tasks, Express.js for backend tasks, and MongoDB with Mongoose for database tasks. No other database is permitted.
4. GitHub repository must be public and named exactly: itue301-exam-[roll-number]-[batch] (e.g. itue301-exam-24it045-A).
5. Use a .env file for the MongoDB connection string and PORT. Do not hardcode credentials. Commit a .env.example; never commit the real .env.
6. Backend must start with node server.js or npm start. Document this in README.md.
7. AI code-generation tools are permitted for this exam (see AI Tools row above). Regardless of how the code was produced, you must be able to explain it in viva: code you cannot explain will be marked zero for that task.

8. Sharing code with another student is strictly prohibited. Copying another student's code results in a zero for both students.
9. Include a README.md with project name, frontend/backend run instructions, and MongoDB setup steps.
10. Final submission must include **both** of the following before the examination window closes: (a) your public GitHub repository link together with the final commit SHA, and (b) the PDF report named as specified in your set. Submit both via the Google Form shared by faculty. Late submissions lose 1 mark per 5 minutes beyond a 5-minute grace period.

SET F: EventHub Ticket Booking System

ITUE301 · Open-Book Practical Examination · CSPIT, CHARUSAT · 24/08/2026

Scenario: EventHub wants to move local event ticketing off paper coupons and spreadsheets. Attendees should browse and book event tickets online; organizers manage events; admin oversees the platform.

Data Entities:

- **Attendee**: name, email, phone
- **Event**: title, category, date, venue, ticketsAvailable
- **Ticket**: attendeeId (ref), eventId (ref), quantity, bookingDate, status (booked | cancelled | used)

User Roles: AttendeeOrganizerAdmin

Mandatory Requirements

- React frontend with at least three distinct pages (Login, Events, MyTickets).
- Client-side routing: Events and MyTickets must be protected routes.
- A reusable EventCard component that accepts props and renders event details.
- Global auth state via useState / useContext or Zustand.
- Express backend exposing at least 5 REST endpoints at /api/v1/.
- All protected routes pass through a custom authGuard middleware; global requestLogger applied.
- MongoDB collections for Attendee, Event, and Ticket with Mongoose schemas.
- Schema-level validation (required, enum, min) on all three models.
- Lazy-loaded route for an Organizer Panel using React.lazy + Suspense.
- API call from React using fetch or Axios; display a loading state while data is fetched.

4 Marks

Task 1: React Component Architecture

Create the basic React frontend for the EventHub Ticket Booking System.

Create the following page/components: LoginPage, EventsPage, MyTicketsPage, EventCard.

Place reusable components inside a suitable /components folder.

The EventCard component must accept the following props: title, category, date, venue, ticketsAvailable.

Display all five values.

If ticketsAvailable is 0, render a "Sold Out" badge instead of a "Book" button; otherwise render the "Book" button.

Use props to pass event data from the parent component to EventCard.

Detailed UI styling is not required. Focus on component structure, composition, and correct use of props.

> **Hint:** keep the sold-out check as a simple conditional at the top of the component.

4 Marks

Task 2: React Routing and State Management

Configure React Router with the following routes:

Route	Component
/	LoginPage
/events	EventsPage (protected)
/my-tickets	MyTicketsPage (protected)
/organizer	OrganizerPanel (lazy-loaded, role 'organizer' required)

Create a navigation component containing links to all routes.

Navigation must be performed using React Router links without a full-page reload.

In EventsPage, create a booking form containing at least: selected event, quantity.

Use useState to manage the form data.

At least two state values must be used meaningfully. For example: selected event, ticket quantity.

Implement an AuthContext holding { attendee, token, role }. Nest the role check inside ProtectedRoute: redirect to /events if the required role does not match, rather than logging the user out.

> **Hint:** keep the redirect target for a failed role check different from the redirect target for a missing token, so the two failure cases are easy to distinguish in viva.

4 Marks

Task 3: Express REST API + Middleware

Build the Express backend for managing event ticket bookings.

Implement the following REST endpoints:

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate attendee, issue token
GET	/api/v1/events	Return all events (public)
POST	/api/v1/tickets	Book tickets for an event (protected)
GET	/api/v1/tickets/my	Return the attendee's own tickets (protected)
PATCH	/api/v1/tickets/:id/status	Update ticket status (protected)

Create a custom requestLogger middleware that logs [METHOD] [PATH] [TIMESTAMP] for every request, applied globally.

Create an authGuard middleware that validates the Bearer token and returns 401 if missing or invalid, applied to every route except /auth/login and the public events listing.

POST /api/v1/tickets must decrement the event's ticketsAvailable by the booked quantity.

Implement a global error-handling middleware as the last middleware, returning structured JSON instead of a raw error stack.

Use appropriate HTTP status codes: 200, 201, 400, 401, 500 as applicable. Test the APIs using Postman or Thunder Client.

> **Hint:** validate quantity against remaining availability before decrementing: never let the count go negative.

4 Marks

Task 4: REST API Consumption in React

Use the Express API developed in Task 3 from the React frontend.

In EventsPage, retrieve event information using: GET /api/v1/events.

Use fetch or Axios.

Use useEffect() so that the API request is made when the component mounts.

Maintain three states: events, loading, and error.

The page must: 1) Display a loading indicator while the request is in progress. 2) Display an error message if the request fails. 3) Render the events, via EventCard, after a successful request.

The event information must be rendered from the API response and must not be hardcoded in the React component.

Add a client-side search box that filters the already-fetched events by title, without triggering a new API request.

Hint: keep the full events array in state; derive the filtered list with .filter() on render.

4 Marks

Task 5: MongoDB + Mongoose Schema Design and Validation

Create the Mongoose-based implementation for the EventHub database. Connect to MongoDB using Mongoose.

Create the following three schemas.

Schema	Field Requirement
Attendee	name required; email required, unique
Event	title required; category String; date required; venue required; ticketsAvailable min 0
Ticket	attendeeId ref Attendee; eventId ref Event; quantity min 1; bookingDate default Date.now; status enum, default 'booked'

The Ticket status field must use: booked, cancelled, used.

Use Mongoose references for: attendeeId → Attendee, eventId → Event.

POST /api/v1/tickets must validate that ticketsAvailable >= quantity (return 400 if not), create the Ticket, and decrement ticketsAvailable on the Event document.

GET /api/v1/tickets/my must use .populate('attendeeId', 'name email') and .populate('eventId', 'title date venue').

Connect MongoDB using a connection string stored in .env. For example: MONGO_URI=your_mongodb_connection_string.

Demonstrate at least one validation failure, such as requesting more tickets than are available, and return a meaningful JSON error response instead of exposing the raw Mongoose error object.

Hint: check availability with findById before Ticket.create(); update the event with findByIdAndUpdate.

Submission Requirements

Public GitHub repo itue301-exam-[roll-number]-[batch] with /frontend, /backend, README, .env.example. PDF [RollNo]_SetF_Report.pdf with screenshots: (1) EventsPage with event cards, (2) Postman showing 201 on ticket creation, (3) MongoDB showing a saved document.

Final Submission (both required): ① GitHub repository link + final commit SHA · ② [RollNo]_SetA_Report.pdf: Upload pdf to Google Classroom / Microsoft Team Assignment before the exam ends.